```
app.Use(async (HttpContext context, Func<Task> next) =>      ❶
{
    if (context.Request.Path.StartsWithSegments("/ping"))    ❷
    {
        context.Response.ContentType = "text/plain";         ❸
        await context.Response.WriteAsync("pong");           ❸
    }
    else
    {
        await next();                                        ❹
    }
});

app.UseStaticFiles();
```

❶ The Use extension takes a lambda with HttpContext (context) and Func<Task> (next) parameters.

❷ The StartsWithSegments method looks for the provided segment in the current path.

❸ If the path matches, generates a response and short-circuits the pipeline

❹ If the path doesn't match, calls the next middleware in the pipeline—in this case UseStaticFiles()\

If the incoming request starts with the required path segment (`/ping`), the middleware responds and doesn't call the rest of the pipeline. If the incoming request doesn't start with `/ping`, the extension calls the next middleware in the pipeline, with no branching necessary.

With the `Use` extension, you have control of when and whether you call the rest of the middleware pipeline. But it's important to note that you generally shouldn't modify the `Response` object after calling `next()`. Calling `next()` runs the rest of the middleware pipeline, so subsequent middleware may start streaming the response to the browser. If you try to modify the response after executing the pipeline, you may end up corrupting the response or sending invalid data.

WARNING Don't modify the `Response` object after calling `next()`. Also, don't call `next()` if you've written to the `Response.Body`; writing to this `Stream` can trigger Kestrel to start streaming the response